

Capítulo 6

Patrones en la Grilla: Match3

“Hacer un juego simple muy, muy bien es tan gratificante como hacer un juego realmente complicado que queda ejecutado más o menos.”

—Brian Fiete, *Game Developer (Gamasutra)*, 2010

Hasta ahora, la posición de un objeto ha sido un vector continuo en píxeles. Match3 introduce el otro tipo de espacio que aparece constantemente en videojuegos: una **grilla discreta**, donde la posición de una pieza es un par de índices (*fila, columna*), y la posición en píxeles se deriva de esos índices, no al revés. Antes de llegar ahí, sin embargo, Match3 necesita animar –piezas que caen, piezas que se desvanecen al formar una coincidencia– de una forma que ningún capítulo anterior resolvió de manera reutilizable. Por eso este capítulo se detiene primero en dos piezas de infraestructura que lo hacen posible: funciones como valores, y un sistema de temporizadores construido sobre ellas. Al terminar el capítulo, el lector habrá construido Match3 (04-match3 en el repositorio): selección e intercambio de piezas con el mouse, detección de coincidencias, eliminación animada y relleno en cascada.

6.1. Contexto histórico: el nacimiento del género match-3

A diferencia de los demás capítulos de este libro, Match3 no reconstruye un juego comercial concreto, sino un mecanismo –combinar tres o más piezas iguales– que pertenece por igual a decenas de títulos. Por eso aquí interesa la historia del *género* más que la de un producto con marca registrada. Su origen documentado se sitúa en *Shariki*, desarrollado en 1994 por el programador ruso Eugene Alemzhin para MS-DOS, en el que el jugador intercambiaba fichas adyacentes con el único objetivo de formar líneas de tres o más elementos del mismo color, que se eliminaban mientras nuevas piezas caían desde arriba para rellenar los huecos. Ese esquema –intercambio de piezas adyacentes, eliminación por coincidencia y relleno por gravedad– es la plantilla mecánica que definiría al género en las tres décadas siguientes (Juul, 2010).

La popularización masiva, sin embargo, no llegó hasta 2001, cuando el estudio PopCap Games, fundado en 2000 por John Vechey, Brian Fiete y Jason Kapalka, publicó *Bejeweled*, distribuido a través del navegador web. El académico Jesper Juul, en su libro *A Casual*

Revolution: Reinventing Video Games and Their Players (Juul, 2010), sostiene que juegos como *Bejeweled* –sencillos de aprender, jugables en sesiones brevísimas y distribuidos directamente al consumidor sin pasar por el canal editorial tradicional– catalizaron entre 2000 y 2010 la aparición de la industria del juego casual. Esta industria amplió radicalmente la base de jugadores hacia públicos que no se identificaban previamente como tales. El propio Donovan documenta cómo esta vía de distribución permitió a estudios pequeños como PopCap saltarse el modelo editorial tradicional (Donovan, 2010).

Desde el punto de vista del diseño, la simplicidad aparente del match-3 –una única interacción central de intercambiar y combinar piezas sobre una grilla discreta– ha demostrado ser una base extraordinariamente flexible para generar variantes, curvas de dificultad progresiva y sistemas de monetización (mecanismos para generar ingresos dentro del propio juego, típicamente vidas limitadas, power-ups comprables o publicidad). Este es un fenómeno que trabajos académicos más recientes han analizado formalmente en términos de motivación del jugador y diseño de mecánicas (Li, 2024; Omori & Felinto, 2012).

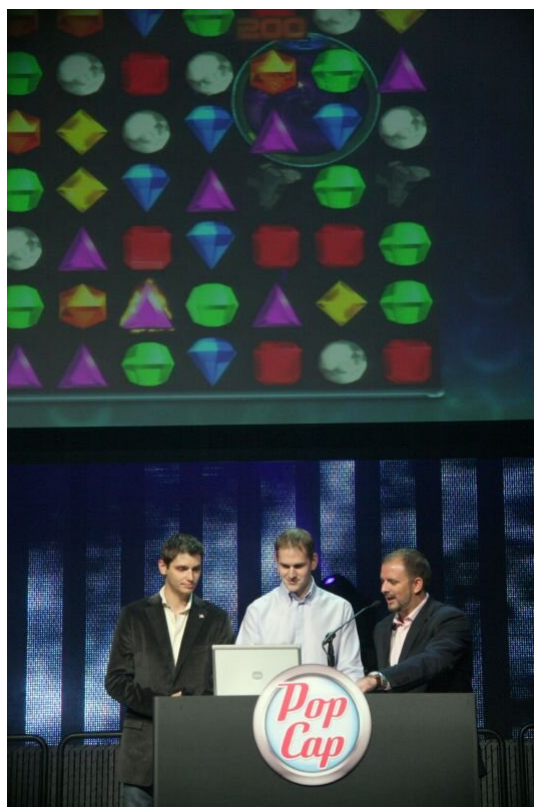


Figura 6.1: John Vechey, Brian Fiete y Jason Kapalka, cofundadores de PopCap Games, en el lanzamiento de *Bejeweled Twist* (2008).

Fotografía de Jon Jordan, licencia CC BY 2.0, vía Wikimedia Commons.

6.2. Funciones anónimas y clausuras: programando el comportamiento

Un temporizador necesita recibir, como argumento, *qué hacer* cuando termine, no un dato sino comportamiento. Antes de construir el sistema de temporizadores de la sección siguiente, conviene aislar el mecanismo de Python que hace eso posible: funciones como valores de primera clase, funciones `lambda`, y clausuras.

6.2.1. Funciones como valores

En Python, una función `def` no es un artefacto sintáctico especial, sino que es un objeto, exactamente como un entero o una cadena. Eso significa que puede asignarse a una variable, guardarse dentro de una lista o un diccionario, y, lo que interesa aquí, pasarse como argumento a otra función, para que esa otra función decida *cuándo* invocarla.

```

1  def saludar():
2      print(";Hola!")
3
4  # La funcion es un valor: se puede asignar, guardar, pasar.
5  accion = saludar
6  acciones = [saludar, accion]
7
8  def ejecutar_luego(funcion):
9      print("Preparando...")
10     funcion()
11
12 ejecutar_luego(saludar) # imprime "Preparando..." y luego ";Hola!"

```

La diferencia entre `saludar` (el objeto función, sin paréntesis) y `saludar()` (el resultado de invocarla) es exactamente la misma diferencia entre pasarle a un temporizador *qué hacer* y pasarle *el resultado de hacerlo ya*. Un temporizador que reciba `saludar()` como argumento ejecutaría el saludo inmediatamente, en el momento de construir el temporizador. Le pasaría, además, `None` (el valor de retorno de `print`) como si fuera la función a invocar después. Este es uno de los errores más comunes al aprender a usar callbacks, y conviene tenerlo presente antes de tocar `gale.timer`.

6.2.2. `lambda`: funciones anónimas de una sola expresión

Cuando el comportamiento a pasar es corto –una sola expresión, usada una sola vez, justo en el lugar donde se necesita– definirla con `def` y ponerle un nombre añade ceremonia sin añadir claridad. Para esos casos, Python ofrece `lambda`: una función anónima cuyo cuerpo es una única expresión, cuyo valor es el retorno implícito.

```

1  cuadrado = lambda x: x * x
2  print(cuadrado(5)) # 25

```

```
3
4 # Equivalente, con nombre:
5 def cuadrado_def(x):
6     return x * x
```

`lambda` conviene cuando la lógica cabe en una línea y se define en el mismo lugar donde se usa: por ejemplo, el **callback** (la función que se pasa como valor para que *otro* código la invoque más tarde, cuando ocurra el evento que le corresponde) `on_finish` de un temporizador, que solo necesita reproducir un sonido o cambiar una bandera booleana. No conviene cuando el cuerpo necesitaría más de una expresión (una `lambda` no admite asignaciones, ciclos, ni múltiples sentencias), o cuando la función se reutiliza en varios lugares y ponerle un nombre documenta su propósito. En esos casos, una función nombrada con `def` sigue siendo la opción correcta, incluso cuando también se pasa como valor.

6.2.3. Clausuras: funciones que recuerdan su entorno

Una **clausura** (*closure*) es una función interna que “recuerda” las variables del entorno donde fue creada, incluso después de que ese entorno, típicamente la función que la contiene, terminó de ejecutarse. Es el mecanismo detrás de un callback que necesita saber, por ejemplo, *cuál* pieza del tablero debe eliminar, sin que esa pieza sea un parámetro explícito del sistema de temporizadores. El temporizador solo sabe invocar una función sin argumentos, así que toda la información que esa función necesita debe venir capturada de su entorno, no recibida como parámetro.

```
1 def hacer_notificador(nombre_pieza):
2     def notificar():
3         print(f"{nombre_pieza} ha sido eliminada")
4     return notificar
5
6 notificar_rojo = hacer_notificador("Pieza roja")
7 notificar_azul = hacer_notificador("Pieza azul")
8
9 notificar_rojo() # "Pieza roja ha sido eliminada"
10 notificar_azul() # "Pieza azul ha sido eliminada"
```

Cada llamado a `hacer_notificador` crea una variable `nombre_pieza` distinta, y la función interna `notificar` la captura por referencia a esa variable concreta, no por su valor en el instante de la creación. Esa distinción –capturar la variable y no una copia congelada de su valor– es inofensiva mientras cada clausura tenga su propia variable, como en el ejemplo anterior. Se vuelve peligrosa, en cambio, cuando varias clausuras comparten la *misma* variable, que es justamente lo que ocurre con una variable de ciclo.

6.2.4. El error clásico: captura tardía de una variable de ciclo

Considérese el intento, aparentemente razonable, de crear una lista de funciones, una por cada pieza de una fila, cada una destinada a imprimir el índice de *su* pieza:

```

1 funciones = []
2 for j in range(4):
3     funciones.append(lambda: print(f"Elimino la pieza en columna
4         {j}"))
5
6 for f in funciones:
7     f()
```

La salida no es 0, 1, 2, 3 como cabría esperar, sino 3, 3, 3, 3. La razón es que las cuatro `lambda` no capturan el *valor* de `j` en el momento en que se crean, sino que capturan la *variable* `j` misma. En Python, una variable de ciclo `for` es una única variable que se reescribe en cada iteración, no una nueva variable por vuelta. Cuando el ciclo termina, `j` vale 3, y las cuatro funciones, al ejecutarse después, leen ese mismo valor final. Esto se conoce como **captura tardía** (*late binding*): la clausura no “fotografía” el valor al crearse, sino que consulta la variable cada vez que se invoca.

La corrección estándar es forzar una captura temprana, dándole a cada clausura su propia variable mediante un parámetro con valor por defecto. Esto funciona porque los valores por defecto sí se evalúan una sola vez, en el momento en que se define la función:

```

1 funciones = []
2 for j in range(4):
3     funciones.append(lambda j=j: print(f"Elimino la pieza en
4         columna {j}"))
5
6 for f in funciones:
7     f()
8
9 # Ahora sí: 0, 1, 2, 3
```

La alternativa igual de común, y a menudo más legible, es evitar la `lambda` y usar una función auxiliar con `def`, que introduce naturalmente un nuevo ámbito por cada llamado:

```

1 def hacer_eliminator(j):
2     return lambda: print(f"Elimino la pieza en columna {j}")
3
4 funciones = [hacer_eliminator(j) for j in range(4)]
```

Este bug –construir varios callbacks dentro de un ciclo, todos con el valor de la última iteración en vez del que le corresponde a cada uno– es exactamente el riesgo que corre el intercambio de piezas de Match3 si sus callbacks se armaran ingenuamente dentro de un ciclo sobre casillas de la grilla. Por eso vale la pena entenderlo a fondo antes de escribir un solo temporizador.

6.3. El tiempo como herramienta: temporizadores, tweens y `gale.timer`

Muchos efectos que Match3 necesita –“la pieza se desvanece antes de desaparecer”, “las piezas de arriba caen suavemente hasta su nueva posición”– requieren una pieza de infraestructura común: algo que sepa ejecutar una función después de cierto tiempo, o interpolar un valor entre dos extremos a lo largo de una duración, sin que cada mecánica del juego tenga que reimplementar su propio reloj interno.

6.3.1. Temporizador de cuenta regresiva: `After` y `Every`

La idea más simple es un temporizador de una sola disparada: acumular Δt en un contador hasta alcanzar una duración fijada, y en ese momento ejecutar una función –la clausura de la Sección 6.2– exactamente una vez. `gale` llama a esto `After`:

```

1 class TimerItemBase:
2     def __init__(
3         self, time: float, on_finish: Optional[Callable[[], None]] = None
4     ) -> None:
5         self.timer: float = 0
6         self.time: float = time
7         self.on_finish: Callable[[], None] = (
8             (lambda: None) if on_finish is None else on_finish
9         )
10        self.to_remove: bool = False
11
12
13 class After(TimerItemBase):
14     def __init__(self, time: float, function: Callable[[], None])
15         -> None:
16         super().__init__(time, on_finish=function)
17
18     def update(self, dt: float) -> None:
19         self.timer += dt
20         if self.timer >= self.time:
21             self.on_finish()
22             self.remove()

```

Nótese que `After` no guarda un reloj propio en segundos absolutos, sino que guarda un contador `self.timer` que empieza en 0 y crece con cada Δt que le llega por `update`. Es exactamente el mismo patrón de acumulación usado para las animaciones de sprites en capítulos anteriores. Cuando el contador alcanza `self.time`, se invoca `on_finish` –la función recibida en el constructor gracias a la clase base `TimerItemBase`– y el objeto se marca con `to_remove = True` para que el sistema lo retire.

Every generaliza esto a una acción que se repite cada cierto intervalo, en vez de dispararse una sola vez, y opcionalmente se detiene después de un número fijo de repeticiones:

```

1 class Every(TimerItemBase):
2     def __init__(
3         self,
4         time: float,
5         function: Callable[[], None],
6         limit: Optional[int] = None,
7         on_finish: Optional[Callable[[], None]] = None,
8     ) -> None:
9         super().__init__(time, on_finish=on_finish)
10        self.function: Callable[[], None] = function
11        self.limit: Optional[int] = limit
12
13    def update(self, dt: float) -> None:
14        self.timer += dt
15
16        if self.timer >= self.time:
17            self.timer %= self.time
18            self.function()
19            if self.limit:
20                if self.limit == 1:
21                    self.on_finish()
22                    self.remove()
23                else:
24                    self.limit -= 1

```

La línea `self.timer %= self.time` es la clave para que un `Every` sin límite se repita indefinidamente sin acumular error. En vez de resetear el contador a 0 (lo que perdería el remanente de Δt que ya sobrepasó la duración), le resta exactamente `self.time` tantas veces como sea necesario, preservando cualquier sobrante para el siguiente ciclo. `Every` es exactamente lo que `Match3` usa para el reloj de la partida, como se verá en la Sección 6.5.

6.3.2. Tween: interpolar atributos de un objeto en el tiempo

Un tween (de *in-between*, “entre cuadros”) es un temporizador que, en lugar de solo ejecutar algo al final, interpola uno o más atributos de uno o más objetos desde su valor inicial hasta un valor objetivo a lo largo de su duración. Para eso usa las funciones de *easing* del Capítulo 1. La implementación de `gale.timer.Tween` es:

```

1 class Tween(TimerItemBase):
2     def __init__(
3         self,
4         time: float,
5         params: Sequence[Tuple[Any, Dict[str, Any]]],
6         ease_function_name: str = "linear",

```

```

7         on_finish: Optional[Callable[[], None]] = lambda: None,
8     ) -> None:
9         super().__init__(time, on_finish=on_finish)
10
11         self.ease_function = EASE_FUNCTIONS.get(
12             ease_function_name)
13
14         if self.ease_function is None:
15             raise RuntimeError(
16                 f"{ease_function_name} is not a valid ease
17                 function for tween"
18             )
19
20         self.plan: Sequence[Tuple[Any, Dict[str, Any]]] = []
21
22         for obj, attrs in params:
23             for key, final in attrs.items():
24                 initial = getattr(obj, key)
25
26                 self.plan.append(
27                     (
28                         obj,
29                         {
30                             "key": key,
31                             "initial": initial,
32                             "final": final,
33                             "change": final - initial,
34                         },
35                     )
36                 )
37
38     def update(self, dt: float) -> None:
39         self.timer += dt
40
41         if self.timer >= self.time:
42             for obj, data in self.plan:
43                 setattr(obj, data["key"], data["final"])
44                 self.on_finish()
45                 self.remove()
46                 return
47
48         for obj, data in self.plan:
49             setattr(
50                 obj,
51                 data["key"],
52                 data["initial"]

```



```

51         + data["change"] * self.ease_function(self.timer
52           / self.time),
    )

```

Hay tres ideas de diseño que vale la pena separar aquí:

- **El plan se calcula una sola vez, al construir el tween:** por cada par (objeto, {atributo: valor_final}) recibido en `params`, el constructor lee el valor *actual* del atributo con `getattr` (ese es el valor inicial) y precalcula `change = final - initial`. De ahí en adelante, `update` no vuelve a leer nada del objeto: solo necesita $t = \text{self.timer} / \text{self.time}$ para reconstruir el valor interpolado como $\text{initial} + \text{change} \cdot \text{ease}(t)$.
- **Un mismo tween puede animar varios atributos de varios objetos a la vez.** Por eso `params` es una secuencia de pares objeto/diccionario. Por ejemplo, el intercambio de dos piezas de Match3 anima `x` e `y` de *dos* `Tile` distintos con un único tween, para que ambas piezas lleguen a su destino exactamente al mismo tiempo.
- **La función de easing es un valor intercambiable,** la misma idea de “comportamiento como parámetro” de la Sección 6.2, aplicada a interpolación en vez de a un evento puntual. `ease_function_name` es una cadena que se busca en el diccionario `EASE_FUNCTIONS` de `gale.ease_functions`; cada una de sus entradas es una función pura $t \rightarrow \text{valor}$, con t normalizado en $[0, 1]$.

La Figura 6.2 muestra tres de estas funciones como curvas $f(t)$ contra t : “`linear`” avanza a ritmo constante, “`in_cubic`” arranca lento y acelera hacia el final, y “`out_cubic`” arranca rápido y desacelera hasta detenerse suavemente. Es la misma forma general de `ease_out_bounce`, solo que sin el rebote. Ver las tres formas lado a lado hace evidente, antes de leer una sola línea de código, por qué la elección de función de easing cambia por completo la sensación de una animación aunque la duración y el valor final sean idénticos.

Por ejemplo, `ease_out_bounce`, usada para dar sensación de rebote a una pieza que cae, se implementa así:

```

1  def ease_out_bounce(t: float) -> float:
2      n1 = 7.5625
3      d1 = 2.75
4
5      if t < 1 / d1:
6          return n1 * t * t
7      elif t < 2 / d1:
8          t -= 1.5 / d1
9          return n1 * t * t + 0.75
10     elif t < 2.5 / d1:
11         t -= 2.25 / d1
12         return n1 * t * t + 0.9375

```

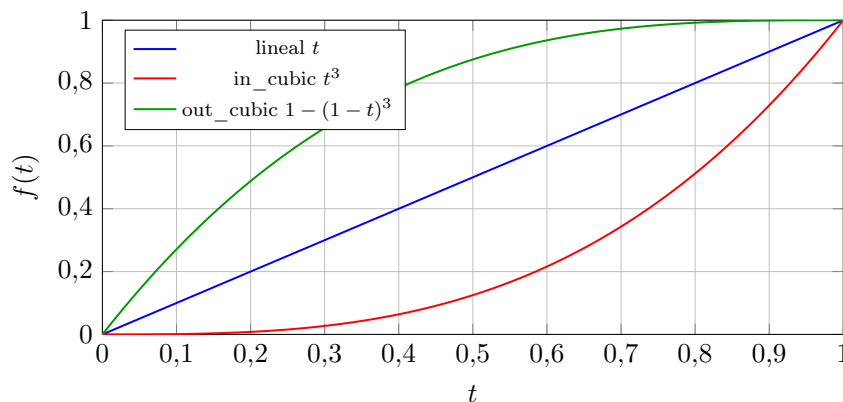


Figura 6.2: Tres funciones de easing como $f(t)$ para $t \in [0, 1]$: la lineal avanza a velocidad constante, la cúbica de entrada (`in_cubic`) acelera progresivamente, y la cúbica de salida (`out_cubic`) desacelera hasta detenerse. La forma de la curva, no solo sus valores en los extremos, determina la sensación de la animación.

```

13     else:
14         t -= 2.625 / d1
15         return n1 * t * t + 0.984375

```

Cambiar de "linear" a `.out_bounce` en el llamado que crea el tween, sin tocar ni una línea de `Tween.update`, basta para cambiar por completo la sensación de la animación. El resto del sistema no sabe ni le importa qué fórmula concreta produce el progreso interpolado.

6.3.3. Encadenamiento de temporizadores

Muchos efectos de Match3 son en realidad una *secuencia* de pasos: primero desvanecer las piezas que forman una coincidencia, y solo *cuando eso termina* hacer caer las piezas de arriba para llenar los huecos. La forma de expresar “cuando esto termine, empieza aquello” con `gale.timer` no es un tipo de dato especial para secuencias, sino simplemente pasar, como `on_finish` de un temporizador, una clausura que crea el *siguiente* temporizador:

```

1 Timer.tween(
2     0.25,
3     [(tile, {"alpha": 0})],
4     on_finish=lambda: Timer.tween(
5         0.25,
6         falling_tiles,
7         on_finish=lambda: resolver_cascada_siguiente(),
8     ),
9 )

```

Esto es preferible a anidar callbacks manualmente porque cada paso sigue siendo una unidad independiente y legible –una duración, un plan de interpolación, un `on_finish`– en

vez de un bloque de lógica monolítico donde “lo que pasa después” está enterrado dentro de “lo que pasa ahora”. Es exactamente la técnica que la Sección 6.5 usa para encadenar la eliminación de una coincidencia con la caída de piezas, y la caída de piezas con la posible detección de una *nueva* coincidencia: una cascada.

6.3.4. Cómo lo implementa gale: la clase Timer

Todo lo anterior –After, Every, Tween– son clases que saben avanzar *a sí mismas* dado un Δt . Falta la pieza que las orquesta: un único lugar donde vivan todos los temporizadores activos del juego, y que el bucle de juego actualice una vez por cuadro. Esa pieza es la clase `Timer`, un *scheduler* a nivel de todo el proceso. Está implementada enteramente con métodos de clase (no hace falta, ni tiene sentido, instanciarla):

```

1 class Timer:
2     items: Union[Every, After, Tween] = []
3     paused: bool = False
4
5     @classmethod
6     def update(cls, dt: float) -> None:
7         if cls.paused:
8             return
9
10        for item in cls.items:
11            item.update(dt)
12
13        cls.items = [item for item in cls.items if not item.
14                     to_remove]
15
16    @classmethod
17    def every(cls, time, function, limit=None, on_finish=None) ->
18        Every:
19        cls.items.append(Every(time, function, limit=limit,
20                               on_finish=on_finish))
21        return cls.items[-1]
22
23    @classmethod
24    def after(cls, time, function) -> After:
25        cls.items.append(After(time, function))
26        return cls.items[-1]
27
28    @classmethod
29    def tween(cls, time, objs, ease_function_name="linear",
30              on_finish=None) -> Tween:
31        cls.items.append(
32            Tween(time, objs, ease_function_name=
33                  ease_function_name, on_finish=on_finish)

```

```

29         )
30         return cls.items[-1]
31
32     @classmethod
33     def clear(cls) -> None:
34         cls.items = []
35         cls.paused = False

```

`Timer.items` es una única lista compartida por todo el juego, sin importar cuántos sistemas distintos hayan pedido un `after`, un `every` o un `tween`: el reloj de cuenta regresiva de la partida, la animación de intercambio de dos piezas y la caída de una columna entera conviven en la misma lista y se actualizan en el mismo recorrido. `Timer.update(dt)` se llama exactamente una vez por cuadro `-gale.game.Game` lo invoca internamente en su bucle principal, así que ningún estado del juego necesita llamarlo a mano. Delega en cada elemento su propio `update(dt)` y después reconstruye la lista filtrando los que se marcaron con `to_remove`. Esta segunda pasada, en vez de eliminar elementos de la lista mientras se recorre, evita el error clásico de modificar una lista durante su propia iteración. `Timer.pause()` y `Timer.resume()` usan el atributo `paused` para congelar *todos* los temporizadores a la vez, útil, por ejemplo, para un menú de pausa. Por su parte, `Timer.clear()` vacía la lista por completo, que es lo que `PlayState` de `Match3` hace al terminar una partida, para que ningún temporizador de una ronda anterior siga ejecutándose en la siguiente.

6.4. Patrones en la grilla

6.4.1. Representación del tablero: una matriz de piezas

`Match3` representa su tablero como una matriz, una lista de listas, de objetos `Tile`, uno por casilla, donde el índice externo es la fila i y el índice interno es la columna j : `self.tiles[i][j]`. Cada `Tile` guarda su propia posición de grilla y su posición en píxeles, calculada una sola vez al crearse:

```

1 class Tile:
2     def __init__(self, i: int, j: int, color: int, variety: int)
3         -> None:
4         self.i = i
5         self.j = j
6         self.x = self.j * settings.TILE_SIZE
7         self.y = self.i * settings.TILE_SIZE
8         self.color = color
9         self.variety = variety

```

La conversión de índice de grilla a píxel es, por tanto, una simple multiplicación por el tamaño de la casilla: $x = j \cdot \text{TILE_SIZE}$, $y = i \cdot \text{TILE_SIZE}$. La conversión inversa, de una posición de mouse en píxeles a un índice de grilla, aparece en `PlayState.on_input`.

Es la división entera correspondiente, restando primero el desplazamiento del tablero en pantalla:

```
1 i = (pos_y - self.board.y) // settings.TILE_SIZE
2 j = (pos_x - self.board.x) // settings.TILE_SIZE
```

La Figura 6.3 hace concreta esta idea antes de verla en código: cada casilla de la grilla tiene un par de índices (i, j) , y su posición en píxeles se obtiene multiplicando esos índices por el tamaño fijo de la casilla, nunca al revés.

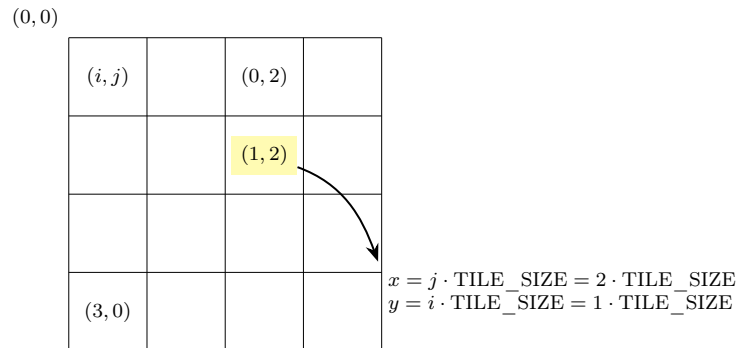


Figura 6.3: Identificación de cada casilla de la grilla por un par de índices (i, j) , fila y columna, independiente de cualquier posición en pantalla. La posición en píxeles de una casilla se *deriva* de sus índices multiplicándolos por `TILE_SIZE`, como en la casilla resaltada $(1, 2)$.

Es importante notar que `x` e `y` *no* se recalculan automáticamente cuando `i` o `j` cambian, ya que son dos pares de atributos independientes que el código debe mantener sincronizados a mano cada vez que una pieza cambia de casilla (al intercambiarse, o al caer). Esa sincronización manual es precisamente lo que un tween anima: en vez de asignar `tile.x` y `tile.y` de golpe a su valor final, se interpolan cuadro a cuadro desde su posición actual hasta la nueva. Mientras tanto, `tile.i` y `tile.j` sí se actualizan de inmediato, porque son datos lógicos de la grilla, no algo que tenga sentido animar.

6.4.2. Detección de patrones: tres o más piezas iguales alineadas

Un patrón (o *match*) es una secuencia de tres o más piezas contiguas, en la misma fila o en la misma columna, con el mismo color. `Board._calculate_match_rec` detecta, para una pieza dada, si participa en un patrón, mirando hasta dos casillas hacia cada uno de los cuatro lados:

```
1 ## Check horizontal match
2 h_match: List[Tile] = []
3
4 # Check left
5 if tile.j > 0:
6     left = max(0, tile.j - 2)
```

```

7     for j in range(tile.j - 1, left - 1, -1):
8         if self.tiles[tile.i][j].color != color_to_match:
9             break
10        h_match.append(self.tiles[tile.i][j])
11
12    # Check right
13    if tile.j < settings.BOARD_WIDTH - 1:
14        right = min(settings.BOARD_WIDTH - 1, tile.j + 2)
15        for j in range(tile.j + 1, right + 1):
16            if self.tiles[tile.i][j].color != color_to_match:
17                break
18        h_match.append(self.tiles[tile.i][j])

```

El mismo patrón de código se repite para `v_match`, intercambiando filas por columnas. Si `h_match` (sin contar la pieza original) tiene al menos dos piezas del mismo color, hay un patrón horizontal de tres o más; lo mismo para `v_match` en vertical. Lo interesante de la implementación real no es solo detectar el patrón de *una* pieza, sino **propagar la búsqueda**: cada pieza que resulta estar en un patrón dispara, recursivamente, la misma búsqueda sobre *sus* vecinas, para capturar patrones en forma de L o T donde una pieza es compartida por una línea horizontal y una vertical a la vez:

```

1 for t in match:
2     match += self._calculate_match_rec(t)

```

Dos conjuntos auxiliares, `self.in_match` y `self.in_stack`, evitan que esta recursión entre en un ciclo infinito (una pieza no vuelve a añadirse a un patrón si ya está en él) y evitan reprocesar una pieza que ya está siendo explorada más arriba en la pila de llamadas. `calculate_matches_for` es el punto de entrada: recibe la lista de piezas que acaban de cambiar (por ejemplo, las dos piezas que el jugador intercambió, o todas las que acaban de caer) y devuelve la lista de patrones encontrados, o `None` si no se formó ninguno.

6.4.3. Relleno por gravedad discreta

Cuando un patrón se elimina, las piezas por encima de cada hueco deben caer para ocuparlo —conceptualmente la misma gravedad del Capítulo 4, pero aplicada a índices enteros de columna en vez de a una posición continua, y animada con los tweens de la Sección 6.3—. `Board.get_falling_tiles` recorre cada columna de abajo hacia arriba. Cuando encuentra un hueco seguido (más arriba) de una pieza, la “desliza” hacia abajo cambiando su índice de fila:

```

1 for j in range(settings.BOARD_WIDTH):
2     space = False
3     space_i = -1
4     i = settings.BOARD_HEIGHT - 1
5
6     while i >= 0:

```

```

7         tile = self.tiles[i][j]
8
9         if space:
10             if tile is not None:
11                 self.tiles[space_i][j] = tile
12                 tile.i = space_i
13                 self.tiles[i][j] = None
14                 tweens.append((tile, {"y": tile.i * settings.
15                     TILE_SIZE}))
16                 space = False
17                 i = space_i
18                 space_i = -1
19             elif tile is None:
20                 space = True
21                 if space_i == -1:
22                     space_i = i
23
24         i -= 1

```

Nótese que el algoritmo actualiza `tile.i`, el índice lógico de grilla, de inmediato, en el mismo paso en que mueve la pieza dentro de la matriz `self.tiles`. Pero *no* toca `tile.y`, la posición en píxeles, directamente: en vez de eso agrega a la lista `tweens` un par `(tile, {"z": tile.i * settings.TILE_SIZE})`, delegando la animación de la caída al mismo mecanismo de tween visto en la sección anterior. Una vez que todas las columnas fueron recorridas, el método completa los huecos que quedaron en las filas superiores creando piezas nuevas con color y variedad aleatorios. Cada una empieza visualmente una casilla más arriba de su posición final (`tile.y -= settings.TILE_SIZE`) para que también entre en cuadro cayendo, en vez de aparecer de golpe:

```

1  for j in range(settings.BOARD_WIDTH):
2      for i in range(settings.BOARD_HEIGHT):
3          tile = self.tiles[i][j]
4          if tile is None:
5              tile = Tile(
6                  i, j,
7                  random.randint(0, settings.NUM_COLORS - 1),
8                  random.randint(0, settings.NUM_VARIETIES - 1),
9              )
10             tile.y -= settings.TILE_SIZE
11             self.tiles[i][j] = tile
12             tweens.append((tile, {"y": tile.i * settings.
13                 TILE_SIZE}))
14
15 return tweens

```

El valor de retorno, `tweens`, es exactamente la lista de pares (objeto, atributos)

que `Timer.tween` espera como `params`. Así, la caída completa de un cuadro entero de la grilla, con un número variable de piezas moviéndose a velocidades y distancias distintas, se anima con un único tween, no con uno por pieza.

6.4.4. Máquina de estados a nivel de interacción

A nivel de interacción del jugador, seleccionar y mover piezas también sigue una secuencia de pasos discretos: seleccionar una primera pieza, seleccionar una segunda pieza adyacente, intentar el intercambio, revertirlo si no forma ningún patrón, o resolver la cascada de eliminaciones resultante si sí lo forma. La Sección 6.5 recorre esta lógica completa, implementada en `PlayState`, con el código real del intercambio, la detección y la cascada, todo encadenado con temporizadores.

6.5. Construcción de Match3



Figura 6.4: Una partida en curso de *Match3*.

6.5.1. El tablero inicial sin coincidencias accidentales

`Board._initialize_tiles` llena la matriz asignando a cada casilla un color aleatorio, pero rechazando cualquier color que generaría de inmediato un patrón de tres en la casilla recién colocada. Solo mira hacia arriba y hacia la izquierda, que son las únicas casillas ya llenas en el momento de generar cada una:

```

1 def _is_match_generated(self, i: int, j: int, color: int) -> bool
  :
2     if (
3         i >= 2
4         and self.tiles[i - 1][j].color == color
5         and self.tiles[i - 2][j].color == color
6     ):

```



```

7         return True
8
9     return (
10         j >= 2
11         and self.tiles[i][j - 1].color == color
12         and self.tiles[i][j - 2].color == color
13     )
14
15 for i in range(settings.BOARD_HEIGHT):
16     for j in range(settings.BOARD_WIDTH):
17         color = random.randint(0, settings.NUM_COLORS - 1)
18         while self._is_match_generated(i, j, color):
19             color = random.randint(0, settings.NUM_COLORS - 1)
20         self.tiles[i][j] = Tile(i, j, color, random.randint(0,
            settings.NUM_VARIETIES - 1))

```

Esto garantiza que la partida siempre empiece con un tablero “estable”, sin patrones ya formados desde el primer cuadro, algo que sería visualmente extraño y que confundiría al sistema de puntaje.

6.5.2. Selección, intercambio y detección, encadenados con tweens

Toda la mecánica de arrastrar dos piezas vive en `PlayState.on_input`, respondiendo al evento `click` definido en `settings.py`. El primer clic guarda la casilla seleccionada. El segundo clic, si cae en una casilla ortogonalmente adyacente (`di` o `dj` igual a 1, pero no ambos, para excluir la diagonal y la misma casilla), desactiva la entrada del jugador y arma un tween que intercambia visualmente las posiciones `x/y` de las dos piezas:

```

1 self.active = False
2 tile1 = self.board.tiles[self.highlighted_i1][self.highlighted_j1]
3 tile2 = self.board.tiles[self.highlighted_i2][self.highlighted_j2]
4
5 def arrive():
6     tile1 = self.board.tiles[self.highlighted_i1][self.
7         highlighted_j1]
8     tile2 = self.board.tiles[self.highlighted_i2][self.
9         highlighted_j2]
10
11     (
12         self.board.tiles[tile1.i][tile1.j],
13         self.board.tiles[tile2.i][tile2.j],
14     ) = (
15         self.board.tiles[tile2.i][tile2.j],
16         self.board.tiles[tile1.i][tile1.j],
17     )

```

```

15     tile1.i, tile1.j, tile2.i, tile2.j = tile2.i, tile2.j, tile1.
        i, tile1.j
16     self._calculate_matches([tile1, tile2])
17
18     Timer.tween(
19         0.25,
20         [
21             (tile1, {"x": tile2.x, "y": tile2.y}),
22             (tile2, {"x": tile1.x, "y": tile1.y}),
23         ],
24         on_finish=arrive,
25     )

```

Aquí se ve, con código real, exactamente la técnica de clausura introducida en la Sección 6.2. `arrive` no recibe `tile1` ni `tile2` como parámetros, ya que el contrato de `on_finish` es una función sin argumentos. En vez de eso, los captura del entorno de `on_input` donde fue definida, y los vuelve a leer por índice de grilla dentro de su propio cuerpo (en vez de cerrar sobre los objetos `tile1/tile2` externos directamente) para mayor claridad sobre qué posición de la matriz se está intercambiando. El tween anima únicamente la posición en píxeles; el intercambio real dentro de la matriz `self.board.tiles`, y la actualización de los índices lógicos `i/j` de cada pieza, ocurre todo de una vez en `arrive`, cuando la animación visual ya terminó.

Si `_calculate_matches` no encuentra ningún patrón con las dos piezas intercambiadas, simplemente reactiva la entrada del jugador (`self.active = True`) sin revertir el intercambio visual. En esta versión del juego, cualquier intercambio válido en adyacencia se permite, y solo si forma coincidencia se cobra puntaje y se dispara la cascada:

```

1  def _calculate_matches(self, tiles: List) -> None:
2      matches = self.board.calculate_matches_for(tiles)
3
4      if matches is None:
5          self.active = True
6          return
7
8      settings.SOUNDS["match"].stop()
9      settings.SOUNDS["match"].play()
10
11     for match in matches:
12         self.score += len(match) * 50
13
14     self.board.remove_matches()
15
16     falling_tiles = self.board.get_falling_tiles()
17
18     Timer.tween(

```

```

19         0.25,
20         falling_tiles,
21         on_finish=lambda: self._calculate_matches(
22             [item[0] for item in falling_tiles]
23         ),
24     )

```

Esta última línea es el corazón de la **cascada**: una vez que las piezas terminan de caer, `_calculate_matches` se vuelve a invocar, a sí misma, mediante la clausura del `on_finish`. Pero esta vez lo hace con las piezas que *acaban* de caer como argumento, no con las que originalmente se intercambiaron. Si esas piezas nuevas forman, a su vez, un nuevo patrón (algo perfectamente posible: una pieza generada al azar puede alinearse con sus vecinas), el ciclo completo –eliminar, hacer caer, volver a comprobar– se repite tantas veces como haga falta, encadenando tweens uno tras otro exactamente como se describió en la Sección 6.3. Esto continúa hasta que una ronda de caída no produce ningún patrón nuevo y `self.active` vuelve a `True`.

⚠ Bloquear la entrada mientras dura una animación encadenada

`self.active` existe por una razón muy concreta: mientras dura una cascada –que puede encadenar varios tweens, uno tras otro, durante más de un segundo– la matriz `self.board.tiles` está en un estado intermedio y transitorio. Si el jugador pudiera seleccionar y intercambiar piezas mientras eso ocurre, dos animaciones independientes terminarían escribiendo la misma casilla de la matriz en momentos distintos, produciendo estados inconsistentes difíciles de reproducir (dependen de en qué instante exacto llegó el click). La regla general, no solo para Match3, es esta: cualquier animación que deje el estado lógico “a medio camino” necesita una bandera equivalente a `self.active` que bloquee la entrada hasta que termine.

6.5.3. El reloj de la partida con `Timer.every`

`PlayState.enter` arma, además, el temporizador de cuenta regresiva del nivel usando `Timer.every` con intervalo de un segundo:

```

1 def decrement_timer():
2     self.timer -= 1
3     if self.timer <= 5:
4         settings.SOUNDS["clock"].play()
5
6 Timer.every(1, decrement_timer)

```

Esto convive, en la misma lista `Timer.items`, con los tweens de intercambio y caída que se crean y destruyen constantemente mientras el jugador juega. El diseño de `gale.timer` –una única lista heterogénea, todos actualizados por el mismo `Timer.update(dt)` cada cuadro– es lo que permite que un reloj de un segundo de intervalo y una animación de un

cuarto de segundo de duración se lleven bien sin que ninguno de los dos sistemas sepa que el otro existe.

6.5.4. El resto del ciclo del juego

`StartState`, `BeginGameState` y `GameOverState` completan el flujo alrededor de `PlayState` –pantalla de título, transición entre niveles con un nuevo `Board` y meta de puntaje creciente, y pantalla de fin de partida–, siguiendo el mismo patrón de máquina de estados de `gale.state` usado desde Breakout. `Match3.init` registra los cuatro estados en un único `StateMachine` y arranca en `"start"`. A diferencia de la máquina de interacción de la Sección 6.2, que vive dentro de `PlayState` y nunca sale de él, esta es la máquina de nivel superior: cada uno de sus estados es una pantalla completa del juego, y sus transiciones están disparadas por temporizadores encadenados o por las condiciones de victoria/derrota de la partida, no por clics sobre el tablero.

La Figura 6.5 dibuja esas cuatro pantallas y las transiciones reales que las conectan. `StartState.on_input` dispara `"start" → "begin"` al confirmar la opción “Start” del menú, después de un tween de un segundo que desvanece la pantalla a blanco. `BeginGameState.enter` encadena tres temporizadores –fade-in, texto del nivel al centro, pausa, texto hacia abajo– y solo al final de esa secuencia llama a `self.state_machine.change("play", ...)`.

Ya dentro de `PlayState.update`, dos condiciones compiten en cada cuadro: si el cronómetro de la partida llega a cero, la transición va a `"game-over"`; si el puntaje alcanza `goal_score` primero, vuelve a `"begin"`, pero con `level = self.level + 1`, para armar un tablero nuevo y repetir la secuencia de introducción con el nivel siguiente. Por último, `GameOverState.on_input` regresa a `"start"` al presionar Enter, cerrando el ciclo.

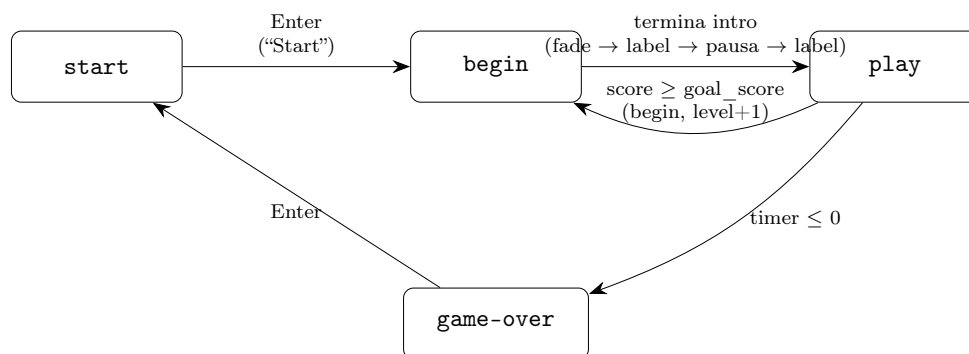


Figura 6.5: Máquina de estados de nivel superior de Match3 (`Match3.state_machine`): `start` pasa a `begin` al confirmar el menú, `begin` pasa a `play` cuando termina su secuencia de temporizadores encadenados, y desde `play` el juego vuelve a `begin` con un nivel más si se alcanza la meta de puntaje, o cae a `game-over` si el cronómetro llega a cero; desde `game-over`, Enter regresa a `start`.

`Match3.update` actualiza además el desplazamiento continuo del fondo, envolviendo el punto de reinicio con el mismo patrón de comparación y reset visto en capítulos anteriores para el scroll de fondo.

Con esto, Match3 queda completo: un tablero de piezas manipulado enteramente a través de índices de grilla, cuya representación visual –incluyendo cada animación de intercambio, desaparición y caída– se resuelve por completo delegando en el sistema de temporizadores construido en la Sección 6.3.

6.6. Ejercicios propuestos

1. Escribir una función `hacer_contador()` que devuelva una clausura que, en cada llamado, retorne el siguiente número entero a partir de 0, sin usar ninguna variable global.
2. Construir, con un ciclo `for` y una lista de `lambda`, el error de captura tardía de variable descrito en la Sección 6.2, comprobar que efectivamente todas las funciones devuelven el mismo valor, y corregirlo con las dos técnicas discutidas (parámetro por defecto y función auxiliar con `def`).
3. Implementar, desde cero y sin usar `gale.timer`, una clase `Temporizador` mínima con `after(duracion, callback)` y un método `update(dt)` que el juego deba llamar cada cuadro, siguiendo el mismo principio de acumulación de Δt que `After`.
4. Extender la detección de patrones de `Board` para reconocer también coincidencias en forma de L o T (más de una línea recta que comparte una pieza), verificando con un tablero de prueba construido a mano que la recursión de `_calculate_match_rec` efectivamente las detecta.
5. Calcular, para un tablero de $n \times n$, cuántas comparaciones en el peor caso hace el algoritmo de detección de coincidencias del capítulo al evaluarse sobre todas las piezas del tablero, y proponer una optimización si n fuera mucho más grande (por ejemplo, limitar la búsqueda a la vecindad de las piezas que cambiaron en el último movimiento, en vez de recorrer el tablero completo).